

Smart Exam System: Maintainer and Supervisor Guide

This document explains the Smart Exam System as it exists in the repository. It is intended to help you present, maintain, debug, and extend the application without depending on an AI assistant.

It covers:

- the architecture and request flow;
- every backend module and important function;
- every API endpoint and its authorization rules;
- the database model and relationships;
- every React route, page, component, form, and state flow;
- grading, answer matching, reports, and downloads;
- local setup, testing, maintenance, debugging, and development procedures;
- limitations and likely supervisor questions.

1. Project At A Glance

Smart Exam System is a two-part web application:

```
Browser
|
| React + Vite, http://localhost:5173
|
| HTTP JSON requests with JWT Bearer token
v
FastAPI backend, http://localhost:8000
|
| SQLAlchemy ORM
v
SQLite database: backend/exam_system.db
```

Technology stack

Area	Technology	Purpose
Frontend	React 19	Components, forms, and page state
Frontend build	Vite	Development server and production build
Frontend styling	Tailwind CSS 4	Utility classes and layout styling
Frontend routing	React Router	URL-based page navigation and protected routes
Backend	FastAPI	REST API, validation, and dependency injection
Persistence	SQLAlchemy + SQLite	ORM and local database
Authentication	JWT + bcrypt	Token authentication and password hashing
Natural language	NLTK WordNet	Synonym suggestions for gap answers
Documents	ReportLab	PDF result and report generation
Spreadsheets	openpyxl	XLSX user and gradebook exports
Testing	unittest	Matcher regression tests

Source layout

```
smart-vle/
  backend/
    app/
      main.py      API routes and business operations
      models.py    SQLAlchemy database models
      schemas.py   Pydantic request/response models
      database.py  Engine, sessions, and schema repair
      auth.py      Passwords, JWTs, and role checks
      grading.py   Question and submission grading
      matcher.py   Canonical matching and format variants
      synonyms.py  WordNet synonym lookup
    tests/
      test_matcher.py
    requirements.txt
    exam_system.db  Local SQLite database, created at runtime
  frontend/
    src/
      main.jsx     React bootstrap
      App.jsx      Route table
      api.js       API and session helpers
      components/  Shared UI components
      pages/       Route-level screens
    package.json
    vite.config.js
  docs/
    APP_GUIDE.md   This document
```

2. Running The Application

Use two terminals from the repository root.

Backend

PowerShell:

```
Set-Location "C:\Users\Student Acc\smart-vle\backend"
& ".\venv\Scripts\python.exe" -m uvicorn app.main:app --reload --port 8000
```

The project virtual environment is important. Installing packages into another Python installation can make the application appear broken even when the code is correct.

If the virtual environment has not been created:

```
Set-Location "C:\Users\Student Acc\smart-vle\backend"
py -m venv .venv
& ".\venv\Scripts\python.exe" -m pip install -r requirements.txt
& ".\venv\Scripts\python.exe" -c "import nltk; nltk.download('wordnet'); nltk.download('omw-1.4')"
```

Frontend

```
Set-Location "C:\Users\Student Acc\smart-vle\frontend"
npm install
npm run dev
```

Open <http://localhost:5173>.

Useful verification commands

```
# Frontend production compilation
Set-Location "C:\Users\Student Acc\smart-vle\frontend"
npm run build

# Frontend linting
npm run lint

# Backend matcher tests
Set-Location "C:\Users\Student Acc\smart-vle\backend"
& ".\venv\Scripts\python.exe" -m unittest discover -s tests -v

# Confirm backend import and schema initialization
& ".\venv\Scripts\python.exe" -c "from app.main import app; print('backend-import-ok')"
```

The full frontend lint command currently reports older issues in some existing files. Always run targeted lint on files you changed as well as the build.

3. Authentication And Session Flow

Login flow

1. The user enters credentials in `frontend/src/pages/Login.jsx`.
2. The page sends `POST /auth/login?username=...&password=...`
3. FastAPI looks up either a fixed demo account or a student registration number.
4. The backend creates a JWT containing:
 - `sub`: the user ID;
 - `role`: the role value;
 - `exp`: eight hours from creation.
5. The frontend stores `access_token`, `role`, and display name in `localStorage` under:
 - `token`;
 - `role`;
 - `name`.
6. The page navigates to the dashboard associated with the role.
7. `apiFetch()` adds `Authorization: Bearer <token>` to protected requests.

Fixed demo accounts

Login name	Password	Role
Lecturer	lecturer101	lecturer
Coordinator	coordinator101	coordinator
HOD	hod101	hod
Admin	admin101	admin

Students are normally created through `/register` with a 14-digit registration number and then log in with that registration number and password.

Important student-account limitation

The admin dashboard can create a user with role `student`, but the current admin form only asks for name, email, password, and role. The backend login flow for students searches by `reg_number`, not by email. Therefore, an admin-created student has no registration number and cannot use the normal student login flow.

This explains the common situation:

- student is created successfully in Admin;
- login fails even though the password is correct.

The reliable current workflow is to use the public student registration form. A future improvement would add `reg_number` to `UserCreate` and the admin form, or allow student login by email.

Authorization flow

`backend/app/auth.py` contains the authentication dependencies:

- `hash_password()` hashes passwords with `bcrypt`;
- `verify_password()` checks a plain password against a `bcrypt` hash;
- `create_access_token()` creates the JWT;
- `get_current_user()` decodes the token and loads the database user;
- `require_lecturer()` and `require_student()` enforce one role;

- `require_role()` creates a dependency accepting one or more roles.

Authorization is enforced by the backend, not merely by hiding buttons in React. A user can inspect frontend code, so every sensitive route must remain protected server-side.

Frontend session behavior

`frontend/src/api.js` is the session utility module:

- `getToken()`, `getRole()`, and `getName()` read `localStorage`;
- `clearSession()` removes all three values;
- `logout()` clears the session and replaces the current route with `/`;
- `apiFetch()` sends JSON and authorization headers;
- `downloadFile()` requests a protected file and triggers a browser download.

`Header.jsx` synchronizes the visible user button with route changes and browser storage events.

`ProtectedRoute.jsx` prevents access to dashboards without a token or with the wrong role.

4. Database Model

All models are in `backend/app/models.py`. IDs are UUID strings generated by `gen_id()`.

User

Stores account identity and authorization data:

- `id`: primary key;
- `name`, `email`;
- `password_hash`: bcrypt hash, never the plain password;
- `role`: lecturer, student, coordinator, hod, or admin;
- `reg_number`: unique nullable student registration number;
- `admission_year`: derived from the first two registration-number digits;
- `is_active`: account can be disabled without deletion.

Course

Stores a unique course code and name. One course can have many exams.

Exam

Stores:

- lecturer owner;
- optional course;
- title;
- start and end timestamps;
- duration in minutes;
- status: pending, approved, or rejected.

An exam has many questions and submissions.

Question

A question belongs to one exam and contains:

- type: mcq, gap, or matching;
- prompt;

- order;
- points.

Question-specific data is stored in child tables:

- AcceptedAnswer for gap answers;
- McqOption for choices and correctness;
- MatchingPair for left/right pairs.

Submission

Stores one student attempt for one exam:

- submitted answers as JSON;
- automatic score and maximum score;
- per-question correctness as JSON;
- optional overridden score and note;
- submission timestamp.

The backend rejects a second submission for the same student and exam.

Announcement

Stores an author, title, body, and creation time. Lecturers and admins can create announcements; authors and admins can edit or delete them; all authenticated roles can read them.

Schema repair

`ensure_database_schema()` runs when `main.py` is imported. It:

1. inspects existing tables;
2. creates missing tables;
3. adds missing columns to existing tables;
4. calls SQLAlchemy `create_all()` for remaining table creation.

This repaired earlier SQLite drift such as missing `users.is_active` and `exams.course_id`. It is a lightweight local repair, not a replacement for Alembic migrations. It does not safely handle every schema change, such as renaming or changing a column type.

5. API Reference

All routes are declared in `backend/app/main.py`. Protected routes require an `Authorization: Bearer <JWT>` header unless stated otherwise.

Health and authentication

Method and path	Access	Purpose
GET /	Public	Returns a simple server-running message
POST /auth/register	Public	Registers a student using <code>reg_number</code> and <code>password</code> query parameters
POST /auth/login	Public	Logs in a fixed account or student registration number
GET /me	Authenticated	Returns the current user

Registration rules:

- registration number must contain exactly 14 digits;

- admission year is 2000 + int(first two digits);
- password must contain at least six characters;
- registration number must be unique.

User administration

Method and path	Access	Purpose
POST /admin/users	admin	Create a user with name, email, password, and role
GET /admin/users	admin	List all users
PATCH /admin/users/{user_id}	admin	Change name, role, or active state
DELETE /admin/users/{user_id}	admin	Delete a user if no linked records prevent it
GET /admin/users.xlsx	admin	Download users as XLSX

Deleting a user with exams or submissions can fail because related records reference that user. The current backend returns HTTP 409 and recommends deactivation. Deactivation is usually safer because it preserves academic history.

Courses

Method and path	Access	Purpose
POST /courses	admin	Create a course
GET /courses	authenticated	List courses
PATCH /courses/{course_id}	admin	Update course code and name
DELETE /courses/{course_id}	admin	Delete a course with no linked exams

Exams and calendar

Method and path	Access	Purpose
POST /exams	lecturer	Create a pending exam owned by the lecturer
GET /exams/pending	coordinator	List exams awaiting approval
PATCH /exams/{exam_id}	owner lecturer or coordinator	Edit exam data; owner cannot edit approved exams
PATCH /exams/{exam_id}/status	coordinator	Approve or reject an exam
DELETE /exams/{exam_id}	owner lecturer, coordinator, or admin	Delete an exam
GET /calendar?start=...&end=...	authenticated	List exams overlapping a time range, scoped by role
GET /exams/{exam_id}	authenticated	Return exam details and role-appropriate question data

Calendar scope:

- lecturers see their own exams;
- students see approved exams only;
- coordinators, HODs, and admins see all exams.

Student exam access additionally checks approval status and the start/end window. Privileged users can inspect exams outside the active window.

Questions

Method and path	Access	Purpose
POST /questions/suggest-answers	lecturer	Generate formatting variants and WordNet synonym senses
POST /exams/{exam_id}/questions/mcq	owner lecturer	Add an MCQ with at least one correct option
POST /exams/{exam_id}/questions/gap	owner lecturer	Add a gap question with accepted answers
POST /exams/{exam_id}/questions/matching	owner lecturer	Add a matching question with at least two pairs
DELETE /questions/{question_id}	owning lecturer	Delete a question

`_get_owned_exam()` is the common ownership check used before adding questions.

Submissions and grading

Method and path	Access	Purpose
GET /exams/{exam_id}/my-submission	student	Read the current student's result
POST /submissions	student	Submit one exam attempt and grade it immediately
GET /exams/{exam_id}/submissions	owner lecturer, coordinator, HOD, admin	List submission summaries
PATCH /submissions/{submission_id}/override	owner lecturer or admin	Replace the displayed score with an approved override and note
GET /submissions/{submission_id}/result.pdf	owning student, owner lecturer, coordinator, HOD, admin	Download result slip

Submission rules:

- exam must be approved;
- current UTC time must be within the exam window;
- one student cannot submit twice;
- all questions must be graded immediately by the backend;
- override scores must be between zero and the submission maximum.

Reports and downloads

Method and path	Access	Purpose
GET /reports/exam/{exam_id}	owner lecturer, coordinator, HOD, admin	Return submission count, average, max score, and pass rate
GET /reports/exam/{exam_id}/summary.pdf	same as report	Download a PDF report
GET /exams/{exam_id}/gradebook.xlsx	owner lecturer, coordinator, HOD, admin	Download submission gradebook

The pass-rate rule is currently 50% or more of maximum score. If a score override exists, reports use the override instead of the automatic score.

Announcements

Method and path	Access	Purpose
POST /announcements	lecturer or admin	Create an announcement
GET /announcements	authenticated	List newest announcements first
PATCH /announcements/{announcement_id}	author or admin	Edit an announcement
DELETE /announcements/{announcement_id}	author or admin	Delete an announcement

6. Backend Module Reference

`main.py`

This is the application composition and business-route module.

Important helper functions:

- `parse_admission_year(reg_number)`: derives the student admission year;
- `validate_reg_number(reg_number)`: validates the 14-digit format;
- `_exam_summary(exam)`: converts an ORM exam into the common API summary shape;
- `_get_owned_exam(exam_id, lecturer, db)`: checks exam existence and lecturer ownership;
- `_compute_exam_report(exam, db)`: calculates report statistics.

Important route operations:

- registration and login create or authenticate users;
- admin operations manage users and courses;
- exam routes implement ownership and approval workflows;
- calendar and detail routes apply role-dependent visibility;
- question routes persist type-specific child data;
- submission routes enforce time, approval, and one-attempt rules;
- PDF/XLSX routes build files in memory and return `StreamingResponse`;
- announcement routes implement author/admin permissions.

`models.py`

Defines SQLAlchemy tables, enums, foreign keys, relationships, and cascade behavior. When adding a field:

1. add it to the model;
2. add it to the relevant Pydantic schema;
3. update route creation/update logic;
4. update frontend request and response handling;
5. migrate the database safely;
6. test old and new records.

`schemas.py`

Pydantic models validate incoming JSON and shape outgoing data. Examples:

- `ExamCreate` validates exam creation fields;
- `McqQuestionCreate`, `GapQuestionCreate`, and `MatchingQuestionCreate` validate question payloads;
- `SubmissionCreate` accepts the flexible answers JSON object;
- `UserOut` prevents password hashes from being returned.

`grading.py`

`grade_question(question, student_answer)` delegates by question type:

- MCQ compares the submitted option ID to the correct option ID;
- gap calls `is_match()` against accepted answers;
- matching verifies every left-to-right pair and requires the same pair count.

`grade_submission(db, exam, answers)` loops over ordered questions, calculates maximum points, records a boolean result for every question, and adds points for correct answers.

`matcher.py`

`canonicalize()` trims, lowercases, removes selected punctuation, and collapses whitespace.

`generate_variants()` creates common name/format variants, including:

- punctuation around initials;
- first and last name forms;
- initials forms;
- punctuation-free forms.

`_answer_variants()` combines generated format variants with WordNet synonym suggestions. `is_match()` canonicalizes the submitted answer and checks whether it is in the generated accepted set.

The lecturer must approve suggested answers in the React form before the gap question is stored. This is important because blindly accepting every synonym could create false positives.

`synonyms.py`

`suggest_synonyms(answer, pos=None)` queries NLTK WordNet and returns synonym groups containing:

- WordNet sense ID;
- definition;
- alternative lemma names.

WordNet is general English. It does not understand every technical, local, or course-specific term.

`database.py`

- engine points to the local SQLite file;
- `SessionLocal` creates database sessions;
- Base is the SQLAlchemy declarative base;
- `get_db()` yields and closes a session for each request;
- `ensure_database_schema()` repairs missing local tables/columns.

`auth.py`

Centralizes password and JWT behavior. The current JWT secret is hard-coded for a demonstration project. A deployed system must read it from an environment variable or secret manager and rotate it safely.

7. Frontend Structure And React Behavior

Bootstrap and routing

`main.jsx` creates the React root, imports global CSS, enables `StrictMode`, and wraps the application in `BrowserRouter`.

`App.jsx` wraps all routes in `Layout` and defines:

- `/`: Login;
- `/register`: Register;
- `/lecturer-dashboard`: lecturer-only dashboard;
- `/student-dashboard`: student-only dashboard;
- `/coordinator-dashboard`: coordinator-only dashboard;

- /hod-dashboard: HOD-only dashboard;
- /admin-dashboard: admin-only dashboard;
- /create-exam: lecturer-only exam builder;
- /exam/:id: any authenticated role with role-dependent detail behavior.

CalendarPage.jsx exists as a standalone calendar screen but is not currently registered in App.jsx. Adding a route is required before users can navigate to it.

Shared components

Layout.jsx

Places Header, the route content, and Footer into a full-height column layout.

Header.jsx

Reads the current token/name, displays the VLE title, computes initials, opens a user menu, and calls logout().

The session synchronization effect re-reads localStorage when the pathname changes and listens for browser storage events. Note that a storage event does not fire in the same tab that made the localStorage change; route changes and explicit state updates are still important.

ProtectedRoute.jsx

Reads token and role state. It redirects unauthenticated users to / and users with the wrong role to /. This is a user-experience guard, not a replacement for backend authorization.

Calendar.jsx

- stores the visible month in viewDate;
- requests /calendar for that month's start and end;
- groups returned exams by day;
- renders a Monday-first calendar grid;
- highlights today;
- navigates to /exam/{id} when an event is clicked;
- optionally displays a New event button for lecturers.

Timeline.jsx

Used currently on the student dashboard. It requests upcoming calendar events for 7, 30, or 90 days, filters by title, sorts by date, and links each event to its exam page. Empty or failed requests show a message.

AnnouncementsBoard.jsx

Loads announcements on mount. Lecturers and admins see title/body inputs and can post or delete announcements. All authenticated users see the feed.

Footer.jsx

Displays the application copyright/developer footer.

Pages and their functions

Login.jsx

State:

- username, password: controlled form values;
- error: API or validation message;
- loading: disables the submit button while waiting.

On submit it creates URL query parameters, calls `/auth/login`, stores the returned token/role/name, and navigates to the role dashboard.

Register.jsx

Validates a 14-digit registration number and six-character minimum password in the browser, calls `/auth/register`, shows success/error state, and redirects to login after success.

Client validation improves usability, but backend validation remains authoritative.

AdminDashboard.jsx

Loads users and courses on mount. Its form state is held in `newUser` and `newCourse` objects. Functions:

- `loadUsers()` and `loadCourses()` refresh tables;
- `createUser()` posts the user form;
- `toggleActive(user)` sends a partial update;
- `deleteUser(id)` deletes a user;
- `createCourse()` posts a course;
- `deleteCourse(id)` deletes a course.

It also downloads the users XLSX and renders announcements.

CreateExam.jsx

This is a two-stage wizard.

Stage one stores title, dates, duration, and course. `handleCreateExam()` converts browser `datetime-local` values to ISO strings and creates a pending exam.

Stage two chooses a question type and stores type-specific form state:

- MCQ: options plus one selected correct option;
- gap: canonical answer, generated variants, selected WordNet senses, and approved answers;
- matching: left/right pair rows.

Important functions:

- `handleCreateExam()` creates the exam;
- `addMcqOption()`, `updateMcqOption()`, `removeMcqOption()` manage MCQ rows;
- `suggestAnswers()` calls the suggestion endpoint;
- `toggleSense()` adds/removes all synonyms in a selected sense;
- `removeApprovedAnswer()` removes one approved gap answer;
- `addMatchingPair()`, `updateMatchingPair()`, `removeMatchingPair()` manage pairs;
- `resetQuestionForm()` resets type-specific state;
- `handleAddQuestion()` validates and posts the selected question type.

The exam remains pending until a coordinator approves it.

ExamDetail.jsx

Loads exam details and, for students, attempts to load the student's existing submission. For privileged users it also loads submissions and the report.

Answer state is one object keyed by question ID:

```
{
  questionId: "selected-option-id",
  gapQuestionId: "typed answer",
  matchingQuestionId: {
    Left: "Right"
  }
}
```

Important functions:

- `load()` fetches role-appropriate details;
- `decideExam()` approves or rejects an exam;
- `submitOverride()` sends a lecturer/admin score override;
- `setMcqAnswer()`, `setGapAnswer()`, and `setMatchingAnswer()` update answer state;
- `submitExam()` checks that every question is answered and posts the submission.

The API deliberately hides correct MCQ options, accepted gap answers, and matching pairs from students. Privileged roles receive the answer data.

`StudentDashboard.jsx`

Displays the logged-in name, Timeline, Calendar, and AnnouncementsBoard.

`LecturerDashboard.jsx`

Displays the logged-in name, a Calendar with a New exam button, and announcements.

`CoordinatorDashboard.jsx`

Loads `/exams/pending`, displays pending exams, navigates to their details for approval/rejection, and renders Calendar and announcements.

`HODDashboard.jsx`

Renders Calendar and announcements. HOD users can open exams to inspect reports and download files.

`CalendarPage.jsx`

A separate styled calendar page component. It is currently not connected to a route.

8. How Forms Handle Data

React forms use controlled inputs. The input value comes from state, and `onChange` updates that state.

Typical flow:

```
input event
-> setState()
-> user clicks action
-> validation
-> JSON.stringify(state)
-> apiFetch()
-> FastAPI/Pydantic validation
-> SQLAlchemy database operation
-> JSON response
-> setState() refresh or navigation
```

Examples:

- login uses query parameters because the backend function declares username and password as scalar parameters;
- admin forms send JSON request bodies;
- exam dates are converted from local browser values to ISO timestamps;
- question forms send different payloads based on question type;
- student answers are collected in a single nested object and submitted once;
- downloads use a binary response rather than JSON.

When adding a form:

1. define initial state;
2. bind every field's value and `onChange`;
3. validate required and domain-specific values in the UI;

4. repeat validation in the backend;
5. show loading, success, and error states;
6. clear or refresh state after success;
7. test invalid input, unauthorized input, and duplicate actions.

9. End-To-End Business Workflows

Create and approve an exam

1. Lecturer logs in.
2. Lecturer opens Create Exam.
3. Frontend posts exam details.
4. Backend saves status pending.
5. Lecturer adds questions.
6. Coordinator loads /exams/pending.
7. Coordinator opens the exam and calls the status endpoint.
8. Backend changes status to approved or rejected.
9. Approved exams become visible to students in Calendar.

Take and grade an exam

1. Student opens an approved exam during its time window.
2. Backend omits answer keys from the response.
3. React stores selected answers locally in component state.
4. Student submits once.
5. Backend checks approval, time, and duplicate submission.
6. `grade_submission()` evaluates every question.
7. Submission and score are stored.
8. Student sees the result and can download the PDF.

Review and override a grade

1. Lecturer opens the exam.
2. Backend returns submission summaries.
3. Lecturer enters an override score and note.
4. Backend validates the score range and ownership.
5. Reports and PDFs use the overridden score as the final score.

Reports and exports

Reports are calculated from stored submissions. PDFs and spreadsheets are generated in memory, so the server does not need to create temporary output files.

10. Maintenance Procedures

Daily development routine

1. Pull or inspect changes before editing.
2. Start one backend process on port 8000.

3. Start one frontend Vite process on port 5173.
4. Confirm the backend root endpoint responds.
5. Log in with a known account.
6. Reproduce a change through the UI.
7. Run targeted tests, lint, and build.
8. Inspect `git diff` before handing over.

Avoid starting duplicate Uvicorn processes. A stale process can serve old code and produce misleading CORS or schema errors.

Adding a new API endpoint

1. Define a Pydantic request/response schema if needed.
2. Add the route in `main.py` near its related feature.
3. Add the correct authentication dependency.
4. Check ownership and role authorization explicitly.
5. Validate input and return meaningful HTTP statuses.
6. Add the frontend API call through `apiFetch()`.
7. Add loading and error states in the page.
8. Test success, invalid input, unauthenticated access, wrong-role access, and missing IDs.

Adding a database field

1. Add the SQLAlchemy column to `models.py`.
2. Decide whether it is nullable and whether it needs a default.
3. Add it to schemas and route logic.
4. Update exports and frontend displays if needed.
5. Use a real migration for shared or production databases.
6. Test both a fresh database and an existing database.

For this local project, `ensure_database_schema()` may add simple missing columns, but do not rely on it for destructive or complex migrations.

Adding a new question type

The change crosses several layers:

1. add a `QuestionType` enum value;
2. add a child model/table if the type needs special data;
3. add request schema;
4. add creation route;
5. add `grade_question()` logic;
6. add privileged and student representations in `get_exam_detail()`;
7. add form state and rendering in `CreateExam.jsx`;
8. add answer state and validation in `ExamDetail.jsx`;
9. add tests for correct, incorrect, incomplete, and unauthorized cases.

Database reset for a disposable demo

Stop the backend before removing the local database. From backend:

```
Remove-Item .\exam_system.db
```

The tables will be recreated when the application starts. This destroys all users, exams, questions, submissions, and announcements. Never do this on a real deployment.

Security maintenance before deployment

The following must be changed before production use:

- move SECRET_KEY out of source code;
- use HTTPS;
- configure CORS from environment-specific allowed origins;
- use a production database and migrations;
- add rate limiting and account lockout for login;
- add stronger password policy and password reset;
- avoid exposing detailed errors to untrusted clients;
- add audit logs for grade overrides and administrator actions;
- validate upload/download access and file names if file uploads are added;
- consider secure, HttpOnly cookies instead of localStorage JWTs;
- add database backups and restore testing.

11. Debugging Playbook

Browser says "Failed to fetch"

1. Check whether the backend is running on port 8000.
2. Open the browser Network tab and identify the exact request.
3. Check the backend terminal for the HTTP status.
4. Distinguish:
 - CORS error: browser blocked the response;
 - 401: missing/expired/invalid token;
 - 403: valid user but wrong role, window, or status;
 - 404: wrong route or ID;
 - 409: duplicate or linked record conflict;
 - 500: backend exception or database issue.
5. Confirm the frontend and backend origins match the CORS list.
6. Confirm only one backend process owns port 8000.

Login fails for a newly created student

Check whether the user has a `reg_number`. Current student login searches by registration number. An admin-created student currently has no registration number because the admin form does not collect one. Use public registration or extend the user-create contract.

Delete user fails

The user probably owns exams or has submissions. The database protects linked academic records. Deactivate the user instead, or implement a carefully designed archival policy. Do not casually cascade-delete academic history.

"No such column" SQLite error

This means the database file predates the current model. Stop duplicate servers, confirm the active backend is running from backend/.venv, and inspect the schema. For a disposable demo, reset the database. For retained data, use a proper migration.

Header displays an old user

Inspect localStorage keys token, role, and name. Use the logout menu or clear the session. The header now resynchronizes on navigation and storage events, but authentication state should ideally be held in a shared React context for a larger application.

A student sees an answer key

Check `get_exam_detail()` in `main.py`. The `is_privileged` branch may be leaking fields. Students should receive option IDs/text only for MCQ, left/right item lists without solution pairing for matching, and no accepted answers for gaps.

12. Supervisor Questions And Strong Answers

Why FastAPI and React?

FastAPI provides typed request validation, automatic OpenAPI documentation, dependency injection, and good performance for a REST API. React provides reusable components and controlled form state for the multiple role-specific screens.

Why is authorization in the backend as well as the frontend?

Frontend checks only control navigation and visibility. A user can call an API directly, so the backend must independently authenticate every protected request and enforce role and ownership rules.

How is exam integrity protected?

The backend checks exam approval and time window, prevents a second submission for the same student/exam pair, hides answer keys from students, and stores the submitted answers and score.

How does automatic grading work?

MCQs compare option IDs, matching questions require an exact complete mapping, and gap questions canonicalize text and compare it against lecturer-approved format variants and selected synonym expansions.

Why does the lecturer approve synonyms?

WordNet can return multiple meanings. Lecturer approval prevents unrelated synonyms from being accepted and keeps grading aligned with the intended course context.

What happens if automatic grading is wrong?

An authorized lecturer for that exam or an admin can submit an override score and note. The override is retained separately from the original automatic score, so the original result is auditable in the database.

Why is SQLite used?

SQLite is simple and appropriate for a local academic demonstration. It requires no separate database server. A multi-user production deployment should use PostgreSQL or another server database with formal migrations and backups.

What is the difference between authentication and authorization?

Authentication proves who the user is through a JWT. Authorization decides what that authenticated role or account owner may do. `get_current_user()` handles authentication; `require_role()` and route ownership checks handle authorization.

Why use Pydantic schemas if SQLAlchemy models already exist?

SQLAlchemy models describe persistence. Pydantic schemas describe and validate the API contract. Keeping them separate prevents database-only fields such as `password_hash` from accidentally being returned to clients.

How are dates handled?

The browser sends ISO timestamps after converting `datetime-local` inputs. The backend stores Python datetime values and compares them with `UTC datetime.utcnow()`. A production system should standardize timezone-aware datetimes explicitly and display them in the user's timezone.

What are the current technical limitations?

- demo credentials are fixed for non-student roles;
- student admin creation does not currently provide a registration number;
- JWT secret is hard-coded;
- SQLite schema repair is not a complete migration system;
- no real-time notifications;
- WordNet is not domain-specific;
- frontend authentication is `localStorage`-based;
- automated test coverage is currently narrow, focused on answer matching;
- some existing frontend lint findings remain;
- `CalendarPage.jsx` is not currently connected to a route.

13. Development Checklist

Before implementing a change:

- identify the owning backend route and frontend page/component;
- identify the database entities involved;
- define the authorization rule;
- identify the success and failure responses;
- check whether existing records need migration.

During implementation:

- preserve existing role boundaries;
- validate at both frontend and backend;
- avoid returning sensitive fields;
- keep API response shapes consistent;
- keep loading and error states visible;
- avoid changing unrelated code.

Before completion:

- run the relevant backend tests;
- run targeted frontend lint;
- run `npm run build`;
- manually test the affected role workflow;
- test unauthorized and invalid requests;
- inspect the database if the change involves persistence;

- review the diff and update this guide if behavior changed.

14. Recommended Next Improvements

In a sensible order:

1. Add `reg_number` to admin-created student accounts or support email login.
2. Add formal Alembic migrations and stop relying on ad hoc SQLite repair.
3. Move JWT configuration and CORS origins to environment variables.
4. Create automated API tests for each role and business rule.
5. Introduce a shared React authentication context instead of reading `localStorage` in individual components.
6. Fix all existing frontend lint errors and convert repeated loading functions to stable effect patterns.
7. Add audit history for status changes, user deactivation, deletion attempts, and grade overrides.
8. Standardize timezone-aware datetime handling.
9. Replace fixed demo accounts with a real account provisioning or SSO integration.
10. Add domain-specific answer aliases and lecturer-managed synonym dictionaries.

15. Source Of Truth

When documentation and code disagree, inspect the code in this order:

1. `backend/app/main.py` for route behavior;
2. `backend/app/auth.py` for security dependencies;
3. `backend/app/models.py` for persisted fields and relationships;
4. `backend/app/schemas.py` for API contracts;
5. the relevant React page/component for user interaction;
6. `frontend/src/api.js` for request and session behavior;
7. tests and the README for intended examples.

A good maintainer does not assume that a button, a README statement, or a hidden route proves that an operation is secure. Trace the request from the browser, through the API dependency and database query, to the response and the rendered state.